Universidad Mariano Gálvez de Guatemala
Ingeniería en Sistemas de Información y Ciencias de la Computación
Alumno: Oscar David Cucul González
Carnet: 9941-25-16053
Fecha: 8 de noviembre de 2026

Caso práctico: Alquiler de canchas deportivas

Spring Boot, Spring Data JPA, MySQL y Lombok

1. Base de datos y configuración

Se creó en MySQL la base de datos **db_canchas**. El proyecto Spring Boot 3.3.5 (Java 17, Maven) usa las dependencias Spring Data JPA, MySQL Driver y Lombok. El archivo **application.properties** configura la conexión a MySQL y la opción **spring.jpa.hibernate.ddl-auto=update**, que permite a JPA crear y actualizar las tablas.

```
CREATE DATABASE IF NOT EXISTS db_canchas;
```

application.properties

```
spring.application.name=canchas
spring.datasource.url=jdbc:mysql://localhost:3306/db canchas?createDatabaseIfNotExist=true&useSSL=false&
serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=********
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=false
```

pom.xml (dependencias)

```
<dependencies>
<dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-data-
jpa</artifactId></dependency>
<dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-
j</artifactId><scope>runtime</scope></dependency>
<dependency><groupId>org.projectlombok</groupId><artifactId>lombok</artifactId><optional>true</optional>
</dependency>
</dependencies>
```

2. Entidades JPA y Lombok

Las tres entidades usan **@Entity**, **@Id**, **@GeneratedValue** y las anotaciones de Lombok **@Getter**, **@Setter**, **@NoArgsConstructor** y **@AllArgsConstructor**. Las referencias de la reservación (idCliente, idCancha) se almacenan como identificadores Long, sin relaciones.

Cliente.java

```
package com.canchas.entidad;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @ToString
public class Cliente {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idCliente;
    private String nombre;
    private String apellido;
    private String telefono;
}
```

Cancha.java

```
package com.canchas.entidad;

import jakarta.persistence.*;
import lombok.*;
import java.math.BigDecimal;
```

```
@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @ToString
public class Cancha {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idCancha;
    private String nombre;
    private String tipo;
    private BigDecimal precioHora;
    private Boolean disponible;
}
```

Reservacion.java

```
package com.canchas.entidad;

import jakarta.persistence.*;
import lombok.*;
import java.time.LocalDate;

@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @ToString
public class Reservacion {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idReservacion;
    private Long idCliente;
    private Long idCancha;
    private LocalDate fecha;
    private Integer horas;
}
```

3. Repositorios

Cada repositorio extiende **JpaRepository** con el tipo de la entidad y su clave primaria (Long).

ClienteRepository.java

```
package com.canchas.repositorio;

import com.canchas.entidad.Cliente;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ClienteRepository extends JpaRepository<Cliente, Long> {
}
```

CanchaRepository.java

```
package com.canchas.repositorio;

import com.canchas.entidad.Cancha;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CanchaRepository extends JpaRepository<Cancha, Long> {
}
```

ReservacionRepository.java

```
package com.canchas.repositorio;

import com.canchas.entidad.Reservacion;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ReservacionRepository extends JpaRepository<Reservacion, Long> {
}
```

4. Interfaces y servicios

Para cada entidad hay una interfaz de servicio y su clase de implementación, con las operaciones: guardar, listar, buscar por ID, actualizar y eliminar por ID, utilizando los repositorios.

IClienteServicio.java

```
package com.canchas.servicio;
```

```
import com.canchas.entidad.Cliente;
import java.util.List;
import java.util.Optional;

public interface IClienteServicio {
    Cliente guardar(Cliente entidad);
    List<Cliente> listar();
    Optional<Cliente> buscarPorId(Long id);
    Cliente actualizar(Long id, Cliente datos);
    void eliminar(Long id);
}
```

ClienteServicio.java

```
package com.canchas.servicio;

import com.canchas.entidad.Cliente;
import com.canchas.repositorio.ClienteRepository;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class ClienteServicio implements IClienteServicio {

    private final ClienteRepository repositorio;

    public ClienteServicio(ClienteRepository repositorio) {
        this.repositorio = repositorio;
    }

    @Override
    public Cliente guardar(Cliente entidad) {
        return repositorio.save(entidad);
    }

    @Override
    public List<Cliente> listar() {
        return repositorio.findAll();
    }

    @Override
    public Optional<Cliente> buscarPorId(Long id) {
        return repositorio.findById(id);
    }

    @Override
    public Cliente actualizar(Long id, Cliente datos) {
        Cliente existente = repositorio.findById(id)
                .orElseThrow(() -> new RuntimeException("Cliente no encontrado: " + id));
        existente.setNombre(datos.getNombre());
        existente.setApellido(datos.getApellido());
        existente.setTelefono(datos.getTelefono());
        return repositorio.save(existente);
    }

    @Override
    public void eliminar(Long id) {
        repositorio.deleteById(id);
    }
}
```

ICanchaServicio.java

```
package com.canchas.servicio;

import com.canchas.entidad.Cancha;
import java.util.List;
import java.util.Optional;

public interface ICanchaServicio {
    Cancha guardar(Cancha entidad);
    List<Cancha> listar();
    Optional<Cancha> buscarPorId(Long id);
    Cancha actualizar(Long id, Cancha datos);
    void eliminar(Long id);
```

```
}
```

CanchaServicio.java

```
package com.canchas.servicio;

import com.canchas.entidad.Cancha;
import com.canchas.repositorio.CanchaRepository;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;

@Service
public class CanchaServicio implements ICanchaServicio {

    private final CanchaRepository repositorio;

    public CanchaServicio(CanchaRepository repositorio) {
        this.repositorio = repositorio;
    }

    @Override
    public Cancha guardar(Cancha entidad) {
        return repositorio.save(entidad);
    }

    @Override
    public List<Cancha> listar() {
        return repositorio.findAll();
    }

    @Override
    public Optional<Cancha> buscarPorId(Long id) {
        return repositorio.findById(id);
    }

    @Override
    public Cancha actualizar(Long id, Cancha datos) {
        Cancha existente = repositorio.findById(id)
                .orElseThrow(() -> new RuntimeException("Cancha no encontrado: " + id));
        existente.setNombre(datos.getNombre());
        existente.setTipo(datos.getTipo());
        existente.setPrecioHora(datos.getPrecioHora());
        existente.setDisponible(datos.getDisponible());
        return repositorio.save(existente);
    }

    @Override
    public void eliminar(Long id) {
        repositorio.deleteById(id);
    }
}
```

IReservacionServicio.java

```
package com.canchas.servicio;

import com.canchas.entidad.Reservacion;
import java.util.List;
import java.util.Optional;

public interface IReservacionServicio {
    Reservacion guardar(Reservacion entidad);
    List<Reservacion> listar();
    Optional<Reservacion> buscarPorId(Long id);
    Reservacion actualizar(Long id, Reservacion datos);
    void eliminar(Long id);
}
```

ReservacionServicio.java

```
package com.canchas.servicio;

import com.canchas.entidad.Reservacion;
import com.canchas.repositorio.ReservacionRepository;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;
```

```
@Service
public class ReservacionServicio implements IReservacionServicio {

    private final ReservacionRepository repositorio;

    public ReservacionServicio(ReservacionRepository repositorio) {
        this.repositorio = repositorio;
    }

    @Override
    public Reservacion guardar(Reservacion entidad) {
        return repositorio.save(entidad);
    }

    @Override
    public List<Reservacion> listar() {
        return repositorio.findAll();
    }

    @Override
    public Optional<Reservacion> buscarPorId(Long id) {
        return repositorio.findById(id);
    }

    @Override
    public Reservacion actualizar(Long id, Reservacion datos) {
        Reservacion existente = repositorio.findById(id)
                .orElseThrow(() -> new RuntimeException("Reservacion no encontrado: " + id));
        existente.setIdCliente(datos.getIdCliente());
        existente.setIdCancha(datos.getIdCancha());
        existente.setFecha(datos.getFecha());
        existente.setHoras(datos.getHoras());
        return repositorio.save(existente);
    }

    @Override
    public void eliminar(Long id) {
        repositorio.deleteById(id);
    }
}
```

5. Pruebas con CommandLineRunner

La clase **PruebasRunner** implementa CommandLineRunner y ejecuta en orden las seis pruebas solicitadas, mostrando los resultados en consola.

PruebasRunner.java

```
package com.canchas;

import com.canchas.entidad.*;
import com.canchas.servicio.*;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
import java.math.BigDecimal;
import java.time.LocalDate;

@Component
public class PruebasRunner implements CommandLineRunner {

    private final IClienteServicio clienteServicio;
    private final ICanchaServicio canchaServicio;
    private final IReservacionServicio reservacionServicio;

    public PruebasRunner(IClienteServicio c, ICanchaServicio ca, IReservacionServicio r) {
        this.clienteServicio = c;
        this.canchaServicio = ca;
        this.reservacionServicio = r;
    }

    @Override
    public void run(String... args) {
        System.out.println("\n===== 1. Registrar 2 clientes y 2 canchas =====");
        Cliente c1 = clienteServicio.guardar(new Cliente(null, "Oscar", "Cucul", "5555-1111"));
```

```
        Cliente c2 = clienteServicio.guardar(new Cliente(null, "Ana", "López", "5555-2222"));
Cancha ca1 = canchaServicio.guardar(new Cancha(null, "Cancha 1", "Fútbol 5", new
            BigDecimal("150.00"), true));
Cancha ca2 = canchaServicio.guardar(new Cancha(null, "Cancha 2", "Baloncesto", new
            BigDecimal("100.00"), true));
        System.out.println("Registrados: " + c1 + " | " + c2 + " | " + ca1 + " | " + ca2);

        System.out.println("\n===== 2. Registrar 2 reservaciones =====");
        Reservacion r1 = reservacionServicio.guardar(
new Reservacion(null, c1.getIdCliente(), ca1.getIdCancha(), LocalDate.now().plusDays(1), 2));
        Reservacion r2 = reservacionServicio.guardar(
new Reservacion(null, c2.getIdCliente(), ca2.getIdCancha(), LocalDate.now().plusDays(2), 1));
        System.out.println("Registradas: " + r1 + " | " + r2);

        System.out.println("\n===== 3. Listar clientes, canchas y reservaciones =====");
        System.out.println("-- Clientes --");
        clienteServicio.listar().forEach(System.out::println);
        System.out.println("-- Canchas --");
        canchaServicio.listar().forEach(System.out::println);
        System.out.println("-- Reservaciones --");
        reservacionServicio.listar().forEach(System.out::println);

        System.out.println("\n===== 4. Buscar una cancha por ID =====");
        canchaServicio.buscarPorId(ca1.getIdCancha())
                .ifPresentOrElse(c -> System.out.println("Encontrada: " + c),
                        () -> System.out.println("No existe la cancha"));

        System.out.println("\n===== 5. Actualizar precio de una cancha =====");
        System.out.println("Antes:   " + canchaServicio.buscarPorId(ca1.getIdCancha()).get());
Cancha cambios = new Cancha(null, ca1.getNombre(), ca1.getTipo(), new BigDecimal("200.00"),
            ca1.getDisponible());
        canchaServicio.actualizar(ca1.getIdCancha(), cambios);
        System.out.println("Después: " + canchaServicio.buscarPorId(ca1.getIdCancha()).get());

        System.out.println("\n===== 6. Eliminar una reservación y mostrar listado =====");
        reservacionServicio.eliminar(r2.getIdReservacion());
        reservacionServicio.listar().forEach(System.out::println);
    }
}
```

CanchasApplication.java

```
package com.canchas;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class CanchasApplication {
    public static void main(String[] args) {
        SpringApplication.run(CanchasApplication.class, args);
    }
}
```

Evidencias

Salida en consola de las seis pruebas



Tabla cliente en MySQL Workbench

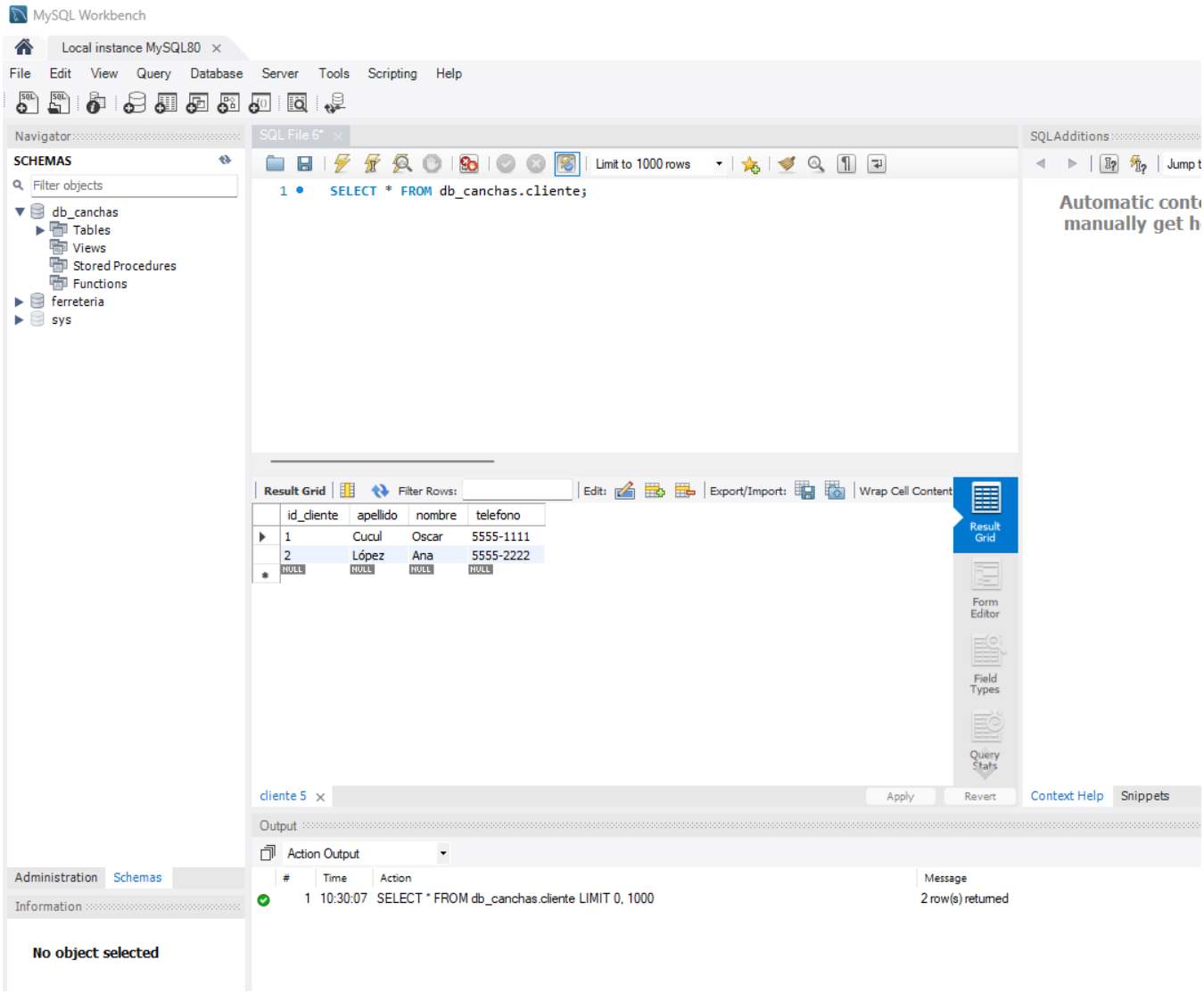


Tabla cancha en MySQL Workbench (precio de la Cancha 1 actualizado a 200.00)

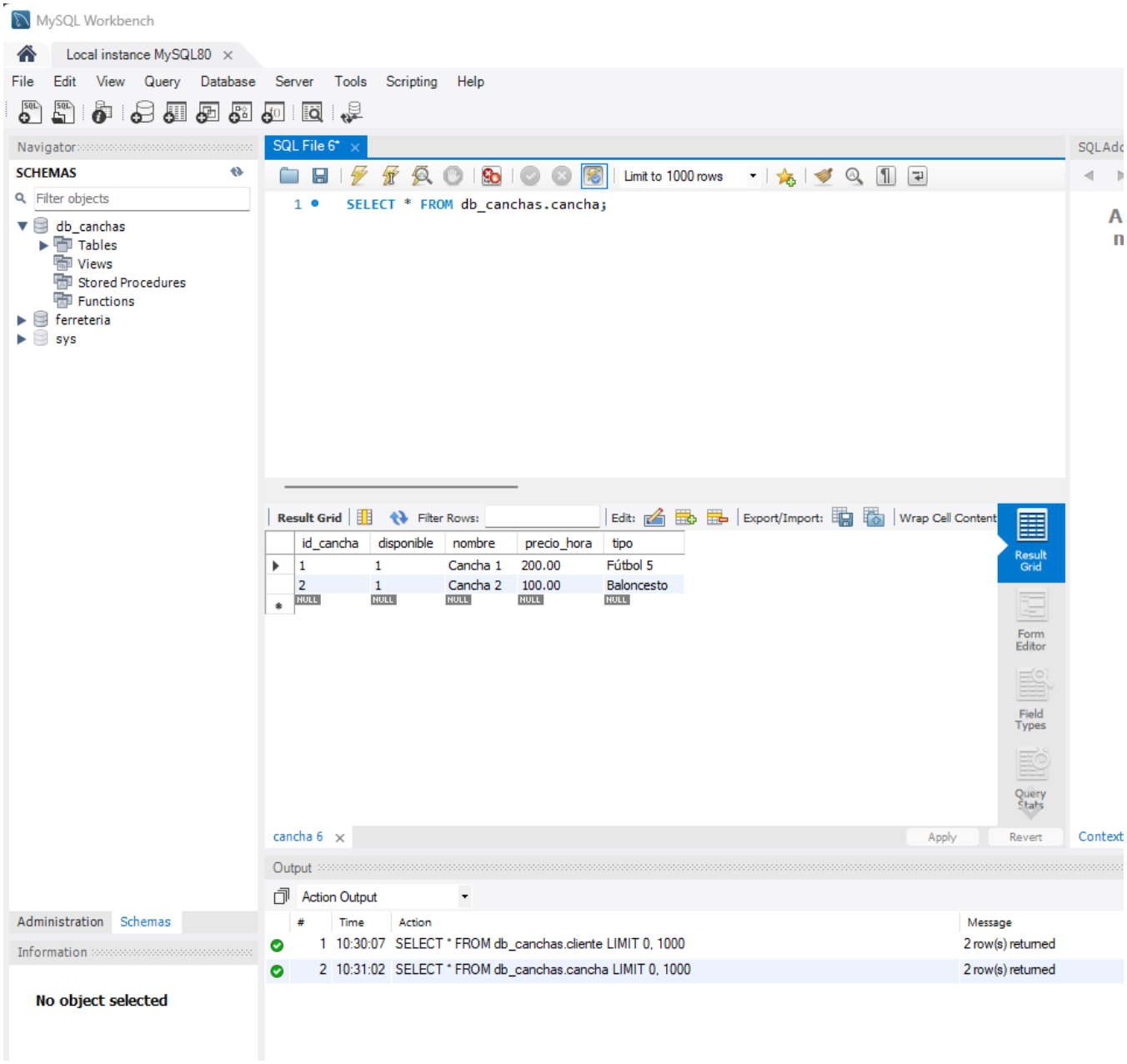


Tabla reservacion en MySQL Workbench

